

CS5491 AI Module A: MDP Cheatsheet

1. Big Picture: MDP 在考什么？

MDP = 已知模型下的序贯决策。每一步 agent 观察 state, 选择 action, 环境按概率转到 next state, 并给 reward。目标不是“下一步分数最高”, 而是长期期望折扣回报最大。
An MDP models sequential decision making with known transition and reward models. The goal is to find a policy maximizing expected discounted return.
和 RL 的区别: MDP 题通常给你 T, R, γ , 让你 planning / compute values; RL 题通常不知道 T, R , 从 samples 学。

MDP = $\langle S, A, T, R, \gamma \rangle$	
S	states: 系统当前情况; grid world 中通常一个格子是一个 state.
A	actions: 每个 state 可采取的动作, 如 up/down/left/right.
T	transition: $T(s, a, s') = P(s' s, a)$, 做 a 后到 s' 的概率。
R	reward: $R(s, a, s')$ 或 $R(s')$, 一步即时奖励。
γ	discount factor: $0 \leq \gamma \leq 1$, 未来奖励折扣。

Policy / Value / Q-value: 三个最容易混的量
• $\pi(s)$: policy 策略: 给每个 state 指定 action。
A policy specifies which action to take in each state.
• $V(s)$: state value: 从 s 出发的期望折扣总回报。
Expected total discounted reward starting from state s .
• $Q(s, a)$: q-value: 在 s 先做 a , 之后继续行动的期望折扣总回报。
Expected return after taking action a in state s .
• **Immediate reward R 不是 value V** 。 R 只看一步; V, Q 看未来总收益。

2. Markov Property: 判断题怎么写

判断标准: 当前 state 是否已经包含预测未来所需的全部信息? 如果下一步还必须知道过去发生过什么, state 设计就不 Markov。
The next-state distribution and reward should depend only on the current state and action, not on the full history.
满足时写: This process satisfies the Markov property because, given the current state and action, the next-state distribution and reward are independent of the past history.
不满足时写: This process is not Markov because the current state omits information from the history that affects future transitions or rewards.
例子:
• Grid cell as state 通常 Markov: 下一步只由当前格子和动作决定。
• 如果“电池电量/是否拿过钥匙/上一动作方向”会影响未来, 但 state 没包含它们, 则不 Markov。

3. Utility / Discount: 为什么不是只看下一步？

$$U([r_0, r_1, r_2, \dots]) = \gamma^0 r_0 + \gamma^1 r_1 + \gamma^2 r_2 + \dots$$

- γ 大: 重视长期; γ 小: 重视眼前。
- 若 $\gamma \neq 1$, 无限和通常收敛, utility 有限。
- Terminal state: 到终点后没有 future term, 常写后续价值 0。
- Finite horizon: 只算接下来 k 步; infinite horizon: 迭代到收敛。

4. MDP Formulation 题: 照抄结构

Question: Formulate the grid world/problem as an MDP.
Answer skeleton:
• **States S :** all valid grid cells / all possible configurations.
• **Actions A :** up, down, left, right, or the actions given.
• **Transition T :** e.g. intended direction 0.8, perpendicular directions 0.1 each; if blocked, stay put.
• **Reward R :** use given rewards, e.g. green +1, red -1, elsewhere -2.
• **Discount γ :** use the given value; mention terminal states if any.
中文答题顺序: 状态 $\rightarrow$ 动作 $\rightarrow$ 转移概率 $\rightarrow$ 奖励 $\rightarrow$ 折扣 $\rightarrow$ 终止状态。不要一上来写 Bellman, formulation 题先定义五元组。

5. Bellman Equations: 公式和含义

$$V^*(s) = \max_a Q^*(s, a)$$
$$Q^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

直觉: 每个 action 先算“可能后继状态的概率加权收益”, 再选最大。
For each action, compute expected immediate reward plus discounted future value, then take the maximum.
固定策略版本: 如果题目说 evaluate a given policy, 不可以自由选 action, 因此没有 max:
$$V^\pi(s) = \sum_{s'} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V^\pi(s')].$$

由 value 选策略:
$$\pi^*(s) = \arg \max_a \sum_{s'} T(s, a, s') [R + \gamma V^*(s')].$$

6. Reward Convention: 先看题目怎么定义 R

同一类 grid world 可能有两种写法, **考试必须按题目定义用**。
A. Reward depends on transition / arrival:

$$\text{Score}(a) = \sum_{s'} T(s, a, s') \{R(s, a, s') + \gamma V_k(s')\}.$$

B. Reward depends on current state, tutorial 常这样列:

$$\text{Score}(a) = R(s) + \gamma \sum_{s'} T(s, a, s') V_k(s').$$

判断方法: 写 $R(\text{green}), R(\text{red})$ 多半是 state reward; 写 “reward for transition” 用 $R(s, a, s')$ 。

7. Value Iteration: 最常考计算题

$$V_{k+1}(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

初始化常见: $V_0(s) = 0$; 重复到 values 变化很小或题目指定轮数。

- 一题一步模板 / One-step template
- 1. 圈出要更新的 state s 。
 - 2. 枚举每个 action a : up/down/left/right。
 - 3. 对该 action 写出所有可能 s' 和概率。
 - 4. 对每个 action 算:

$$\text{Score}(a) = \sum_{s'} T(s, a, s') \{R + \gamma V_k(s')\}$$

- 5. 取最大: $V_{k+1}(s) = \max_a \text{Score}(a)$ 。
- 算法伪代码:
- initialize $V_0(s) = 0$ for all states.
 - for each iteration k : update every non-terminal state using Bellman optimality update.
 - stop after fixed k iterations or when $\max_s |V_{k+1}(s) - V_k(s)|$ is small.
 - extract policy using $\arg \max_a$.

同步 vs 异步: 标准 VI 用 old V_k 算 new V_{k+1} 。若题目明确是 fixed policy / 给定动作, 才按该动作; 若题目说 value iteration, 仍要对 actions 取 max。

8. Policy Iteration: 和 Value Iteration 怎么区分

Policy iteration = evaluation + improvement.
• 1. Initialize a policy π .
• 2. **Policy evaluation:** 固定 π , 解/迭代 V^π , **不取 max**。
• 3. **Policy improvement:** 用 V^π 做 one-step look-ahead, 选更好的 action。
• 4. 如果 policy 不再变化, 停止。
Value Iteration 每轮直接更新 V ; 每个 state 每轮都 max; 最后 extract policy.
Policy Iteration 先评估当前 policy, 再改 policy; policy 常比 values 更早收敛。

9. 看到题目先分类

“Formulate”	写 S, A, T, R, γ . 不用计算。
“Compute V_{k+1} ”	Value iteration: 枚举 actions, 算期望, 取 max。
“Evaluate policy”	Policy evaluation: 只用 $\pi(s)$ 指定动作, 不取 max。
“Extract policy”	给 V , 对每个 action 做 one-step look-ahead, 取 argmax。
“Markov?”	看 current state 是否包含预测未来所需全部信息。

10. 高频坑 / Common Mistakes

- **Policy evaluation 不取 max; value iteration 才取 max**。
- 漏乘 transition probability T 。
- 漏乘 discount γ , 或把 γ 错乘到 immediate reward 上。
- Terminal state 还继续加未来项; terminal future 通常 0。
- Reward/value 混淆: R 是 immediate, V 是 future expected utility。
- 撞墙/障碍未按题目处理; 通常 stay put 也要算进概率。
- 不区分 old values 和 newly updated values. 题目没说时, 标准 value iteration 用 old V_k 。

11. 英文短答模板

MDP definition:
An MDP consists of states, actions, a transition function, a reward function, and a discount factor. The goal is to find a policy maximizing expected discounted utility.
Value iteration:
Value iteration repeatedly applies the Bellman optimality update. For each state, it evaluates all actions by expected immediate reward plus discounted next-state value and keeps the maximum.
Policy iteration:
Policy iteration alternates between evaluating the current policy and improving it greedily with respect to the computed values, until the policy no longer changes.
Why Q is useful:
If q-values are known, the agent can select an action directly by $\arg \max_a Q(s, a)$ without doing a one-step look-ahead using the transition model.

12. 考场最短流程

- 先写题型: formulation / VI / policy evaluation / extraction / Markov。
- 再写所用公式: 公式先拿分。
- 计算题必须列概率项, 不要只写最终数字。
- 最后一行写 conclusion: chosen action / value / whether Markov。

12A. Exact Policy Evaluation: 方程组模板

若题目要求 exact values under a fixed policy, 要为每个非终止 state 写方程并联立求解:

$$V^\pi(s_i) = \sum_{s'} T(s_i, \pi(s_i), s') \{R + \gamma V^\pi(s')\}.$$

例:

$$V(A) = r_A + \gamma(0.7V(A) + 0.3V(B)),$$
$$V(B) = r_B + \gamma(0.4V(A) + 0.6V(B)).$$

提醒: fixed policy 的 action 已由 $\pi(s)$ 决定, **不要写 max**。

13. Tutorial Grid World 规则

State
Action
Transition
Reward
Discount
Terminal

grid cell, bottom-left origin [0,0]; green [2,3].
up/down/left/right.
intended 0.8; perpendicular 0.1 + 0.1.
green +1, red -1, everywhere else -2.
 $\gamma = 0.8$.
green ends; future term 0.

row 2	[2, 0]	[2, 1]	[2, 2]	[2, 3] G
row 1	[1, 0]	wall?	[1, 2]	[1, 3] R
row 0	[0, 0]	[0, 1]	[0, 2]	[0, 3]

Boundary / wall: 如果动作撞墙或进障碍，通常 stay in same state；若题目另说，按题目。

14. Action Transition: 先把概率画出来

Perpendicular
Right
Left
Up
Down

= 垂直两边。
0.8 right, 0.1 up, 0.1 down.
0.8 left, 0.1 up, 0.1 down.
0.8 up, 0.1 left, 0.1 right.
0.8 down, 0.1 left, 0.1 right.

撞墙/出界处理：把那部分概率加到 “stay in same state”。例如在 bottom row 向 down 的部分若出界，就 $s' = s$ 。

15. Tutorial 算题套路：每题都照这个表

Step 1
Step 2

Step 3
Step 4
Step 5

写当前 state s 和当前可用旧 values V_k .
对每个 candidate action 画 3 个后继: intended + two per-
pendicular.

撞墙/障碍的后继改成 stay put.
写 $\text{Score}(a) = R + \gamma \sum p V_k(s')$.
比较 scores, 最大者是 $V_{k+1}(s)$ ；对应动作是 greedy action.

推荐答题格式：

Score(RIGHT) = $-2 + 0.8[0.8V(\cdot) + 0.1V(\cdot) + 0.1V(\cdot)]$
Score(UP) = $-2 + 0.8[0.8V(\cdot) + 0.1V(\cdot) + 0.1V(\cdot)]$
 $V_{\text{new}}(s) = \max(\text{scores})$

16. Tutorial Round 1: 为什么被更新的是 -2

Round 1 / Trajectory 1

$V([2, 3]) = 1,$ $V(\text{visited non-terminal states}) = -2.$

原因：初始 $V_0(s) = 0$ ，普通格 reward 是 -2 ，future term 是 $\gamma \sum p \cdot 0 = 0$ ，所以被 backup 的普通格会变成 -2 。未更新过的格仍可能保持 0，例如后面 [0,0] 例题里的 $V([0, 1]) = 0$ 。

17. Worked Example A: 更新 [0, 0]

按 **tutorial** 的当前格 **reward** 写法：当前格 reward $R([0, 0]) = -2, \gamma = 0.8$. Bellman backup 比较动作后选 RIGHT: intended 到 [0, 1] (旧值 0)，perpendicular 到 [1, 0] (旧值 -2)，另一个方向出界所以 stay at [0, 0] (旧值 -2)。因此：

$V([0, 0]) = -2 + 0.8(0.8 \cdot 0 + 0.1(-2) + 0.1(-2)).$

 $= -2 + 0.8(-0.4) = -2.32.$

写答案时说明：The best action is RIGHT under the current values, so the updated value is -2.32 .

18. Worked Example B: 更新 [1, 0]

当前格 **reward**: $R([1, 0]) = -2$ 。tutorial 中取 UP 作为最优动作：

$V([1, 0]) = -2 + 0.8(0.8(-2) + 0.1(-2) + 0.1(-2)).$

 $= -2 + 0.8(-2) = -3.60.$

解释句：
The expected next-state value under action UP is the probability-weighted sum of the successor values. The Bellman update adds the immediate reward and the discounted expected future value.

19. 如何算任意格子的动作分数

例：state [0, 2]，action RIGHT。

- intended: [0, 3] with prob 0.8.
- perpendicular up: [1, 2] with prob 0.1.
- perpendicular down: out of grid, usually stay [0, 2] with prob 0.1.

Score(RIGHT) = $-2 + 0.8[0.8V([0, 3]) + 0.1V([1, 2]) + 0.1V([0, 2])]$.

若要 **value iteration**: 再把 LEFT/UP/DOWN 都算一遍，取最大。
若题目给 **policy**: 只算 policy 指定的那个 action。

20. Tutorial Checkpoints: 核对答案用

T1
T2
T3
T4

T5
T6

[2, 3] = 1; visited non-terminal states = -2 ; not-yet-updated states may still be 0.
[0, 0] = -2.32 ; [1, 0] = [2, 0] = [2, 1] = -3.60 ; [2, 2] = -1.52 .
[0, 0] = -2.47 ; [0, 1] = [0, 2] = -2 ; [1, 2] = -2.28 ; [1, 3] = -1 .
[0, 0] = -3.77 ; [0, 1] = -3.60 ; [0, 2] = -2.34 ; [0, 3] = -2.08 ;
[1, 3] = -1 .
[0, 0] = -4.89 ; [1, 0] = [2, 0] = -4.88 ; [2, 1] = -3.54 ;
[2, 2] = -1.66 .
[0, 0] = -5.09 ; [0, 1] = -4.07 ; [0, 2] = -3.70 ; [1, 2] = -3.06 ;
[2, 2] = -1.74 .

注意：这些是 tutorial trajectory 表里的核对值；考试变形时按题目给的 old values, 是否 fixed policy、是否 value iteration、以及 reward convention 重新列式。

21. 变形题常见问法

- “Compute one Bellman update for state s ”: 算所有 action scores, 取 max.
- “Which action is optimal at s ?”: 不一定要完整更新，只需比较 action scores.
- “What if γ becomes smaller?”: 更重视 immediate reward, 远处绿色吸引力下降。
- “What if living reward is more negative?”: agent 更想快点终止，避免走太久。
- “What if transition is deterministic?”: intended prob 1，没有 perpendicular terms.

21A. Tutorial 表的关键理解

- Tutorial 的表按 trajectory 展示更新，某些没被更新过的格子仍可能用初始值 0。
- 因此 [0,0] 例子中 $V([0, 1]) = 0$ ，但 $V([1, 0]) = -2$ 、 $V([0, 0]) = -2$ 。
- 如果考试给完整 old value table，就完全按表代入；不要把 tutorial 的数值硬套到新题。
- 若题目说 “after one full synchronous iteration”，通常所有 states 都用同一张 old table 更新。

22. Policy Evaluation 计算模板

题目给 **fixed policy** π 时：

$$V^\pi(s) = \sum_{s'} T(s, \pi(s), s') \{R + \gamma V^\pi(s')\}.$$

步骤：

- 找 $\pi(s)$ 指定动作，**不要枚举后取 max**。
- 根据该动作列出 successor probabilities。
- 如果是 one-step evaluation，用给定旧 V 代入。
- 如果要求 exact values，通常要为多个 states 建线性方程组一起解。

英文说明：
Policy evaluation computes values under a fixed policy. It does not choose the best action at each update.

23. Policy Extraction 计算模板

题目给 $V(s)$ ，问 **best action / policy**：

$$\pi(s) = \arg \max_a \sum_{s'} T(s, a, s') \{R + \gamma V(s')\}.$$

- 对每个 action 算 one-step look-ahead。
- 最大 score 的 action 就是 policy arrow。
- 如果两个 action 分数相同，可写 tie；若题目给 tie-breaking，按题目。

24. 数字计算自检

- 概率和应为 1: $0.8 + 0.1 + 0.1 = 1$ 。
- 负 reward 的普通格，若周围 future values 也负，更新值通常更负。
- 靠近 green 的格子可能不那么负，因为 +1 被折扣后传回来。
- 如果算出非常大的正数/负数，先检查是否漏了概率或 γ 。
- 保留两位小数通常够；题目要求几位按题目。

25. 考卷英文可直接套

Bellman update answer:
For each possible action, I compute the probability-weighted expected value of the successor states, add the immediate reward and discount the future value. The updated value is the maximum over these action values.
Grid transition sentence:
When the intended action is RIGHT, the agent moves right with probability 0.8 and moves to the two perpendicular directions with probability 0.1 each. If a move hits a wall, the agent remains in the same state.
Terminal sentence:
The terminal state's future value is zero because the episode ends after reaching it.
Policy comparison sentence:
Value iteration updates values directly using a max over actions, whereas policy iteration alternates between evaluating a fixed policy and improving that policy.

26. 最后 30 秒检查

- 我有没有写清楚 S, A, T, R, γ 或用到的 Bellman 公式？
- 我有没有把每个 successor 的概率乘上？
- 我有没有区分 $R(s)$ 和 $V(s)$ ？
- 我有没有在 policy evaluation 误取 max？
- Terminal / wall / out-of-grid 有没有正确处理？
- 最后一行有没有写 final value / final action / conclusion？

27. Wall / Out-of-grid 合并概率例子

在 [2, 0] 选 **UP**: intended up 出界, left 也出界, right 到 [2, 1]:

Score(UP) = $R + \gamma[0.9V([2, 0]) + 0.1V([2, 1])]$.

在 [1, 0] 选 **RIGHT**, 若 [1, 1] 是 wall:

Score(RIGHT) = $R + \gamma[0.8V([1, 0]) + 0.1V([2, 0]) + 0.1V([0, 0])]$.

原则：撞墙/障碍不是丢掉概率，而是把该概率加到 stay put。